

Pattern Matching: *variant-like* and `std::expected`

Document #:	P3527R1 [Latest] [Status]
Date:	2025-01-13
Project:	Programming Language C++
Audience:	Evolution Library Evolution
Reply-to:	Michael Park < mcypark@gmail.com > Zach Laine < whatwasthataddress@gmail.com >

Contents

- [1 Motivation and Scope](#)
 - [1.1 The Variant Protocol](#)
- [2 Design](#)
 - [2.1 Optional additions](#)
 - [2.2 Not included: modifying `std::visit`](#)
 - [2.3 A note about `std::bad\_variant\_access`](#)
- [3 Proposed Wording](#)
- [4 Implementation Experience](#)
- [5 References](#)

§ 1 Motivation and Scope

[\[P2688R3\]](#) introduces pattern matching for C++, including handling of `std::variants`. For example:

```
auto v = std::variant<int32_t, int64_t, float, double>{/* ... */};
v match {
  int32_t: let i32 =>
    std::print("got int32: {}", i32);
  int64_t: let i64 =>
    std::print("got int64: {}", i64);
  float: let f =>
    std::print("got float: {}", f);
  double: let d =>
```

```
std::print("got double: {}", d);  
};
```

However, there is an unsupported type in the standard that is variant-like but is not covered by [P2688R3] without explicit library support. That type is `std::expected`.

This proposal does not interact with the pattern matching language feature, nor does it interact with user defined types; it only affects whether `std::expected` interoperates gracefully with the language feature.

This paper adds library facilities that allow `std::expected` to conform to the notion of variant-like used in the pattern matching core wording. In order to accomplish this, we propose to add an API for `std::expected` that is syntactically and semantically equivalent to the API used by pattern matching to match `std::variants`.

§ 1.1 The Variant Protocol

The language semantics for pattern matching ([P2688R3]) include the notion of a “variant protocol”. This is analogous to the tuple protocol used for structured bindings. The tuple protocol is used in defining the behavior of pattern matching as well.

This section does not describe a library change, and is not part of this proposal. It is here to describe the relevant language part of the pattern matching design, since that language design informs the library design in this paper.

To match based on the currently-engaged value of a variant-like object `v` of type `V`, the pattern matching language feature uses `std::variant_size<V>`, `std::variant_alternative<V>`, `get<I>(v)`, and `v.index().get()`. `get()` is used via ADL-only calls. The template parameter passed to `get()` is a `size_t` NTTP, not a type.

§ 2 Design

The minimal design is this.

- Add an `index()` member function to `std::expected`.
- Add specializations of `std::variant_size`, and `std::variant_alternative` that work with `std::expected`.
- Add `size_t`-parameter overloads of `std::get` that work with `std::expected`.

By making these changes, `std::expected` will be compatible with `match` expressions wherever `std::variant` is.

§ 2.1 Optional additions

The changes above make `std::expected` very close to `std::variant` in many ways, but there are a few ways in which the two differ. Since the main design above brings them into

such near-alignment, it might be useful to add even more compatability, so that the user can treat `std::variant` and `std::expected` as equivalent in most generic code. The optional changes are:

- Add a version of `std::holds_alternative()` that works with `std::expected`.
- Add type overloads of `std::get` that work with `std::expected`.
- Add overloads of `std::get_if` that work with `std::expected`.

The costs of wording and implementing these additions is low, so they seem like good additions for the sake of consistency.

§ 2.2 Not included: modifying `std::visit`

The list of optional additions above leaves out a pretty important API – `std::visit`. These are the reasons the authors chose not to include it.

1. `std::expected` is fixed at two alternatives. No one would write `std::visit` when they can simply write `if`, unless perhaps they were handling variant-like types generically, in which case...
2. ..., the advent of pattern matching as a language feature means we can use that for generic handling of variant-like types. No one has to write `std::visit` ever again, and that's for the better.
3. There was an increased-complexity cost (and associated compile time cost) to implementing `std::visit` support for `std::expected`. This stems from the fact that `std::visit` takes a variadic pack of variants. If some elements of the pack were also allowed to be `std::expecteds`, there would have to be compile-time conditional handling for these two types added to each element of the pack.

The benefits are nearly zero, and the costs are non-zero. Worse still, the costs apply to code that never uses `std::expected` with `std::variant`, which violates the “don't pay for what you don't use” principle.

§ 2.3 A note about `std::bad_variant_access`

Much of the API proposed here is throwing. Each throwing function throws `std::bad_variant_access`, same as the ones that work with `std::variant`. There exists another exception type associated with `std::expected`, `std::bad_expected_access`. However, this is the wrong type to use. `std::bad_expected_access` is thrown when the user attempts to access the value of an `expected`, when that `expected` has an error engaged, not a value.

§ 3 Proposed Wording

Add to the end of 22.8.2 [\[expected.syn\]](#):

```
+ // [expected.helper], expected helper class specializations
+ template<class T, class E>
+   struct variant_size<expected<T, E>>;
+
+ template<size_t I, class T, class E>
+   struct variant_alternative<I, expected<T, E>>;
+
+ // [expected.get], value access
+ template<class U, class T, class E>
+   constexpr bool holds_alternative(const expected<T, E>&) noexcept;
+
+ template<size_t I, class T, class E>
+   constexpr variant_alternative_t<I, expected<T, E>>&
+   get(expected<T, E>&); //
+   freestanding-deleted
+ template<size_t I, class T, class E>
+   constexpr variant_alternative_t<I, expected<T, E>>&&
+   get(expected<T, E>&&); //
+   freestanding-deleted
+ template<size_t I, class T, class E>
+   constexpr const variant_alternative_t<I, expected<T, E>>&
+   get(const expected<T, E>&); //
+   freestanding-deleted
+ template<size_t I, class T, class E>
+   constexpr const variant_alternative_t<I, expected<T, E>>&&
+   get(const expected<T, E>&&); //
+   freestanding-deleted
+
+ template<class U, class T, class E>
+   constexpr T& get(expected<T, E>&); //
+   freestanding-deleted
+ template<class U, class T, class E>
+   constexpr T&& get(expected<T, E>&&); //
+   freestanding-deleted
+ template<class U, class T, class E>
+   constexpr const T& get(const expected<T, E>&); //
+   freestanding-deleted
+ template<class U, class T, class E>
+   constexpr const T&& get(const expected<T, E>&&); //
+   freestanding-deleted
+
+ template<size_t I, class T, class E>
+   constexpr add_pointer_t<variant_alternative_t<I, expected<T, E>>>
+   get_if(expected<T, E>*) noexcept;
+ template<size_t I, class T, class E>
+   constexpr add_pointer_t<const variant_alternative_t<I, expected<T,
+   E>>>
+   get_if(const expected<T, E>*) noexcept;
+
+ template<class U, class T, class E>
+   constexpr add_pointer_t<T>
+   get_if(expected<T, E>*) noexcept;
+ template<class U, class T, class E>
+   constexpr add_pointer_t<const T>
+   get_if(const expected<T, E>*) noexcept;
```

Add a new member function index to 22.8.6.1 [\[expected.object.general\]](#)

§ 22.8.6.1 [\[expected.object.general\]](#) General

```
namespace std {
    template<class T, class E>
    class expected {
    public:
        ...

        // [expected.object.obs], observers
        ...
        template<class U> constexpr T value_or(U&&) const &;
        template<class U> constexpr T value_or(U&&) &&;
        template<class G = E> constexpr E error_or(G&&) const &;
        template<class G = E> constexpr E error_or(G&&) &&;
+   constexpr size_t index() const noexcept;

        ...
    };
}
```

Add a new member function index to 22.8.6.6 [\[expected.object.obs\]](#)

§ 22.8.6.6 [\[expected.object.obs\]](#) Observers

```
+ constexpr size_t index() const noexcept;
```

26 *Returns:* `has_value() ? 0 : 1`.

Add a new member function index to 22.8.7.1 [\[expected.void.general\]](#)

§ 22.8.7.1 [\[expected.void.general\]](#) General

```
template<class T, class E> requires is_void_v<T>
class expected {
public:
    ...

    // [expected.void.obs], observers
    ...
    template<class G = E> constexpr E error_or(G&&) const &;
    template<class G = E> constexpr E error_or(G&&) &&;
+   constexpr size_t index() const noexcept;

    ...
};
```

Add a new member function index to 22.8.7.6 [\[expected.void.obs\]](#)

§ 22.8.7.6 [\[expected.void.obs\]](#) Observers

```
+ constexpr size_t index() const noexcept;
```

15 *Returns:* `has_value()` ? 0 : 1.

Add a new section `[expected.helpers]` after 22.8.7.8 [\[expected.void.eq\]](#):

§ 22.8.7.8+1 `[expected.helpers]` **expected helper class specializations**

```
+ template<class T, class E>
+     struct variant_size<expected<T, E>> : integral_constant<size_t, 2> {
+         };
+
+ template<size_t I, class T, class E>
+     struct variant_alternative<I, expected<T, E>> {
+         using type = see below ;
+     };
```

1 *Mandates:* $I < 2$

2 *Type:* The type is `T` if I is 0. Otherwise, the type is `E`.

Add a new section `[expected.helpers]` after 22.8.7.8 [\[expected.void.eq\]](#):

§ 22.8.7.8+2 `[expected.get]` **Value access**

```
+ template<class U, class T, class E>
+     constexpr bool holds_alternative(const expected<T, E>& e) noexcept;
```

1 *Returns:* `true` if `e.index() == 0` && `e.has_value()` is `true`, or if `e.index() == 1` && `e.has_error()` is `true`. Otherwise, returns `false`.

```
+ template<size_t I, class T, class E>
+     constexpr variant_alternative_t<I, expected<T, E>>&
+         get(expected<T, E>& e);                                //
+         freestanding-deleted
+ template<size_t I, class T, class E>
+     constexpr variant_alternative_t<I, expected<T, E>>&&
+         get(expected<T, E>&& e);                                //
+         freestanding-deleted
+ template<size_t I, class T, class E>
+     constexpr const variant_alternative_t<I, expected<T, E>>&
+         get(const expected<T, E>& e);                            //
+         freestanding-deleted
+ template<size_t I, class T, class E>
+     constexpr const variant_alternative_t<I, expected<T, E>>&&
+         get(const expected<T, E>&& e);                            //
+         freestanding-deleted
```

2 *Mandates:* $I < 2$.

3 *Throws:* `bad_variant_access` if `e.index() != I`.

4 *Returns:* `*e` if `has_value()` is `true`, and `e.error()` otherwise.

```

+ template<class U, class T, class E>
+ constexpr T& get(expected<T, E>& e); //
+     freestanding-deleted
+ template<class U, class T, class E>
+ constexpr T&& get(expected<T, E>&& e); //
+     freestanding-deleted
+ template<class U, class T, class E>
+ constexpr const T& get(const expected<T, E>& e); //
+     freestanding-deleted
+ template<class U, class T, class E>
+ constexpr const T&& get(const expected<T, E>&& e); //
+     freestanding-deleted

```

- 5 *Effects:* If *e* holds a value of type *U*, returns a reference to that value. Otherwise, throws an exception of type `bad_variant_access`.

```

+ template<size_t I, class T, class E>
+ constexpr add_pointer_t<variant_alternative_t<I, expected<T, E>>>
+     get_if(expected<T, E>* e) noexcept;
+ template<size_t I, class T, class E>
+ constexpr add_pointer_t<const variant_alternative_t<I, expected<T,
+     E>>>
+     get_if(const expected<T, E>* e) noexcept;

```

- 6 *Mandates:* $I < 2$.

- 7 *Returns:* A pointer to the value stored in the `expected`, if `e != nullptr` and `v->index() == I`. Otherwise, returns `nullptr`.

```

+ template<class U, class T, class E>
+ constexpr add_pointer_t<T>
+     get_if(expected<T, E>* e) noexcept;
+ template<class U, class T, class E>
+ constexpr add_pointer_t<const T>
+     get_if(const expected<T, E>* e) noexcept;

```

- 8 *Effects:* If *e* holds a value of type *U*, returns a pointer to that value. Otherwise, returns `nullptr`.

§ 4 Implementation Experience

One of the authors implemented all the changes proposed here in a branch of Clang/libc++. The implementation was straightforward, and posed no unforeseen challenges. The changes did not cause any of the pre-existing libc++ tests to fail.

The changes can be found [here](#).

§ 5 References

[P2688R3] Michael Park. 2024-10-16. Pattern Matching: ``match`` Expression.
<https://wg21.link/p2688r3>